

CUDA Chapter 5 Notes: Sections 5.1–5.4

Memory Access Efficiency, CUDA Memory Types, and Tiled Matrix Multiplication

1 The Main Task of Chapter 5

The previous chapters focused mainly on how CUDA organizes and schedules parallel work: grids contain blocks, blocks contain threads, and threads are executed in warps on streaming multiprocessors. Chapter 5 shifts the focus from *how many threads are running* to *how efficiently those threads obtain and reuse data*. The central objective is simple:

Every value fetched from global memory should support as much useful computation as possible.

This matters because global memory is large but relatively expensive. Arithmetic hardware can execute operations very quickly, but it cannot do useful work until the required operands have arrived. Chapter 5 therefore connects memory behavior, computational intensity, the Roofline model, the CUDA memory hierarchy, and finally shared-memory tiling.

2 Latency, Bandwidth, and Why Memory Can Limit Performance

Memory latency is the delay between issuing a memory request and receiving the requested value. A global-memory access may require many clock cycles. GPUs tolerate this delay through **latency hiding**: when one warp is waiting for data, the SM scheduler can issue instructions from another ready warp whose state is already resident on the SM. In this way, a long latency does not necessarily mean that the entire SM must remain idle.

Memory bandwidth, in contrast, is the maximum amount of data that can be transferred per unit time. If a device can deliver at most $B_{\max}$ bytes per second, launching additional warps cannot raise the sustained transfer rate beyond that physical limit. More resident warps can hide latency, but they cannot eliminate a bandwidth bottleneck. Once the memory system is saturated, the only fundamental way to obtain substantially more arithmetic throughput is to **perform more useful work per byte transferred**.

3 Naive Matrix Multiplication and Computational Intensity

Consider

$$P = MN,$$

where M , N , and P are square matrices. One CUDA thread computes one output element

$$P_{row,col} = \sum_{k=0}^{Width-1} M_{row,k} N_{k,col}.$$

For a fixed thread, *row* and *col* remain unchanged while *k* moves through the shared dimension. Therefore, that thread reads **one row of *M* and one column of *N*** and forms their dot product.

In each iteration of the dot product, the thread reads one single-precision value from *M* and one from *N*. Since one `float` occupies 4 bytes, the iteration transfers

$$2 \times 4 = 8 \text{ bytes}$$

from global memory. The same iteration performs one multiplication and one addition, which are counted as

$$2 \text{ FLOPs.}$$

The accumulator is normally kept in a register, so it does not require a global-memory access in every iteration. The final output is also written only once after the complete dot product. The dominant repeated traffic is therefore the two input loads.

The **compute-to-global-memory-access ratio**, also called computational or arithmetic intensity, is

$$I = \frac{\text{useful floating-point operations}}{\text{bytes transferred from global memory}}$$

and is measured in FLOP/B. For the naive matrix-multiplication inner loop,

$$I_{\text{naive}} = \frac{2 \text{ FLOPs}}{8 \text{ B}} = 0.25 \text{ FLOP/B.}$$

This is a very low intensity: a large amount of data movement supports only a small amount of arithmetic.

3.1 A100 example

Using the textbook A100 example with peak memory bandwidth

$$B_{\text{max}} = 1555 \text{ GB/s,}$$

the bandwidth can support at most approximately

$$\begin{aligned} P_{\text{memory}} &= B_{\text{max}} I_{\text{naive}} \\ &= 1555 \times 0.25 \\ &\approx 389 \text{ GFLOP/s.} \end{aligned}$$

This is far below the stated ordinary FP32 peak of about 19.5 TFLOP/s. The important conclusion is not the exact percentage, but the architectural principle:

Powerful arithmetic hardware is useful only when the memory system can supply enough data, or when each fetched value is reused enough times.

4 The Roofline Model

The Roofline model combines the two major upper bounds on kernel performance: the maximum rate at which memory can deliver operands and the maximum rate at which the GPU can execute arithmetic.

The horizontal axis is **computational intensity**, measured in FLOP/B. Moving to the right means that every byte fetched from global memory supports more computation. The

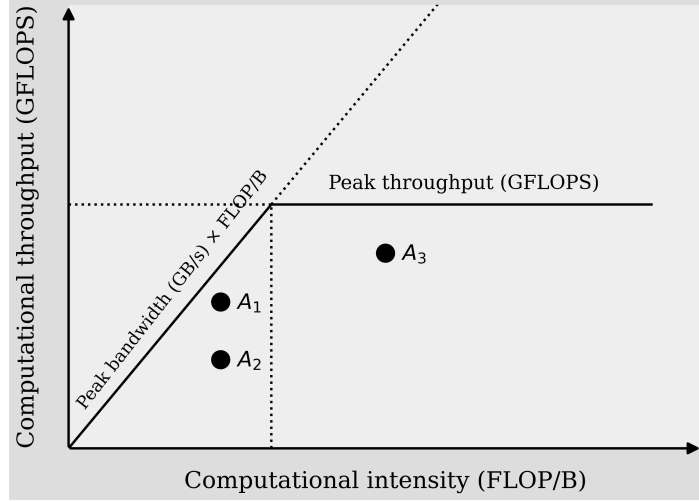


Figure 1: Basic Roofline model. The sloped line is the memory-bandwidth limit and the horizontal line is the peak-compute limit.

vertical axis is **achieved computational throughput**, usually measured in GFLOP/s or TFLOP/s.

The sloped line is the **memory-bandwidth roof**. Its equation is

$$P = B_{\text{peak}} I.$$

At low computational intensity, throughput grows approximately in proportion to I : if each byte is reused more effectively, the same memory bandwidth can support more arithmetic work. This is the **memory-bound region**.

The horizontal line is the **compute roof**,

$$P = P_{\text{peak}}.$$

Once computational intensity is high enough, memory can theoretically feed the arithmetic units fast enough and the upper limit becomes the compute capability of the GPU. This is the **compute-bound region**. Together, the two limits give

$$P_{\text{achievable}} \leq \min(P_{\text{peak}}, B_{\text{peak}} I)$$

The intersection is the **ridge point**, where

$$I_{\text{ridge}} = \frac{P_{\text{peak}}}{B_{\text{peak}}}.$$

The three points in Figure 1 illustrate three different situations. A_1 is **memory bound but close to the sloped roof**, so the kernel is already using available memory bandwidth relatively efficiently; a major improvement requires higher data reuse and therefore higher computational intensity. A_2 is **also memory bound but lies far below the memory roof**; before changing the algorithm, its implementation should first improve memory-access efficiency, such as coalescing, occupancy, or memory-level parallelism. A_3 **lies in the high-intensity region and is mainly compute bound**; its next optimizations should focus on arithmetic utilization rather than simply reducing global-memory traffic.

5 Data Reuse and the Preview of Tiling

Matrix multiplication contains substantial inherent reuse. One element of M participates in several output elements in the same output row, and one element of N participates in several output elements in the same output column. A naive implementation allows different threads to request many of these values repeatedly from global memory.

The optimization target is therefore

load a value from global memory once

↓

store it in fast on-chip memory

↓

reuse it for many arithmetic operations.

Suppose a $T \times T$ thread block computes a $T \times T$ output tile. In one phase, the block loads one $T \times T$ tile from M and one from N , so the global-memory traffic is

$$2T^2 \times 4 = 8T^2 \text{ bytes.}$$

The block has T^2 threads, and each thread performs T multiply-add steps, giving

$$2T^3 \text{ FLOPs.}$$

Thus the idealized intensity becomes

$$I_{\text{tiled}} = \frac{2T^3}{8T^2} = \frac{T}{4} \text{ FLOP/B}$$

instead of 0.25 FLOP/B. The arithmetic workload is not reduced; the improvement comes from **reusing each value after it has been moved into on-chip shared memory**.

6 A Short Performance Reasoning Procedure

A practical Roofline-style analysis can be kept to four questions:

1. How many useful FLOPs does the important loop or kernel perform?
2. How many bytes must be read from or written to global memory?
3. What is the computational intensity $I = \text{FLOPs/global-memory bytes}$?
4. Is the measured kernel limited mainly by $B_{\text{peak}}I$ or by P_{peak} , and how far is it below that relevant roof?

If a kernel is close to the bandwidth roof, the main route forward is usually more data reuse. If it is far below the bandwidth roof, memory accesses are not yet efficient. If it is close to the compute roof, arithmetic execution becomes the primary optimization target.

7 CUDA Memory Types and Their Relationship to the Execution Hierarchy

A CUDA device executes kernels as grids. A grid consists of many thread blocks, and every block contains many threads. **Registers are private to individual threads**, while **shared memory is allocated per block and can be accessed cooperatively by all threads**

in that block. Global memory and constant memory are device-wide spaces: threads from different blocks can access them, and the host can transfer data to or from these device memory spaces using CUDA APIs.

This hierarchy is important because the memory location determines both performance and sharing behavior. The classical von Neumann picture already distinguishes a small register file close to the ALU from a much larger memory system farther away. CUDA makes this distinction especially important: **the register file and shared memory are on-chip resources inside the GPU, while global memory is normally implemented in off-chip DRAM such as GDDR or HBM.** Accessing a register operand can feed an arithmetic instruction directly; obtaining a global-memory operand requires address generation, a memory transaction, and a much longer trip through the memory hierarchy.

7.1 Registers

Registers are the fastest storage normally visible to a CUDA thread. They are on-chip and private to one thread. Loop counters, indices, pointer variables, and scalar accumulators such as `Pvalue` are commonly placed in registers. Their main limitation is capacity: the register file of an SM is finite, so high per-thread register usage can reduce the number of simultaneously resident blocks and warps. If the compiler cannot keep all thread-private values in registers, some values may be spilled into local memory.

7.2 Local memory

Local memory is private in scope but not necessarily local in physical location. The word “local” means that each thread has its own logical data, but local memory is normally backed by the device-memory hierarchy and is therefore much more expensive than registers. Large thread-private arrays, runtime-indexed arrays, stack data, or spilled registers may be placed there. It should not be confused with a small on-chip scratchpad.

7.3 Shared memory

Shared memory is an explicitly managed on-chip memory allocated per block. Every thread in the same block can read or write the same shared-memory array, while different blocks receive independent copies. This makes shared memory the key cooperation space for tiling: threads can cooperatively load a small working set from global memory, synchronize, reuse the data many times, and then overwrite the same shared-memory storage with the next working set. Because one thread may consume data written by another thread, shared-memory algorithms frequently require `__syncthreads()` barriers.

7.4 Global memory

Global memory is large, device-wide, and normally off-chip. It stores the main input and output arrays of many CUDA programs and can be accessed by threads in any block. Its capacity is much larger than registers or shared memory, but its latency is much higher and its total throughput is limited by memory bandwidth. For performance-sensitive kernels, the key question is therefore not whether global memory can hold the data, but how often the same data must be fetched from it.

7.5 Constant memory

Constant memory is a small device-wide read-only space for kernel code. It is cached and is especially effective when threads in the same warp read the same constant address, because the value can be broadcast. It is useful for small coefficient tables or parameters that

are initialized by the host and then reused by many threads without modification. The textbook capacity is 64 KiB.

7.6 One example containing the main memory spaces

The following example is sufficient to show how the declarations map onto the CUDA memory hierarchy:

```
1 __device__ float globalScale;
2 __constant__ float coefficients[256];
3
4 __global__ void memoryExample(
5     const float* input,
6     float* output,
7     int n)
8 {
9     __shared__ float tile[256];
10
11     int tx = threadIdx.x;
12     int tid = blockIdx.x * blockDim.x + tx;
13
14     float value = 0.0f;
15
16     if (tid < n) {
17         tile[tx] = input[tid];
18     }
19
20     __syncthreads();
21
22     if (tid < n) {
23         value = tile[tx]
24             * coefficients[tx]
25             * globalScale;
26
27         output[tid] = value;
28     }
29 }
```

Here, `globalScale` resides in device global memory, `coefficients` resides in constant memory, and the arrays addressed by `input` and `output` are global-memory data. The array `tile` is one shared-memory object per block. The variables `tx`, `tid`, and `value` are private to each thread and are normally stored in registers. This one kernel therefore shows the most important distinction: **global memory stores large device-wide data, shared memory enables block-level cooperation, and registers hold thread-private working values close to the arithmetic units.**

8 Tiled Matrix Multiplication: From Mathematics to GPU Execution to CUDA Code

The previous sections established the performance motivation for tiling: global-memory traffic is expensive, matrix multiplication contains substantial data reuse, and shared memory provides an explicitly managed on-chip space in which a thread block can reuse data. The tiled matrix-multiplication kernel is therefore best understood as a continuous chain from the original matrix equation, to a block-level algorithm, to a GPU execution strategy, and finally to CUDA code.

8.1 1. Ordinary Matrix Multiplication and the Source of Data Reuse

Consider the ordinary matrix product

$$P = MN,$$

where M , N , and P are square matrices of width W . One output element is

$$P_{i,j} = \sum_{k=0}^{W-1} M_{i,k} N_{k,j}$$

so $P_{i,j}$ is the dot product between row i of M and column j of N . A straightforward CUDA implementation assigns one thread to one output element. Thread (i, j) therefore repeatedly loads one element from row i of M and one element from column j of N while k moves through the shared dimension.

The important observation is that neighboring output elements require overlapping input data. For example,

$$\begin{aligned} P_{0,0} &= M_{0,0}N_{0,0} + M_{0,1}N_{1,0} + M_{0,2}N_{2,0} + M_{0,3}N_{3,0}, \\ P_{0,1} &= M_{0,0}N_{0,1} + M_{0,1}N_{1,1} + M_{0,2}N_{2,1} + M_{0,3}N_{3,1}. \end{aligned}$$

Both outputs reuse the complete row $M_{0,:}$. Similarly, outputs in the same column reuse elements from the corresponding column of N . In a naive kernel, different threads may request the same matrix elements from global memory again and again. The mathematical number of multiply-add operations is unavoidable, but the repeated movement of the same input values is not.

This is exactly the optimization opportunity introduced earlier in this chapter:

load a value from global memory once
 $\downarrow$
 keep it in fast on-chip storage
 $\downarrow$
 reuse it for several arithmetic operations.

Tiling reorganizes matrix multiplication around this reuse pattern.

8.2 2. Block Matrix Multiplication

Before considering CUDA, it is useful to rewrite matrix multiplication purely as a block-matrix algorithm. Let the tile width be T , and partition the three matrices into $T \times T$ submatrices. For example, if $W = 4$ and $T = 2$,

$$M = \begin{bmatrix} M^{(0,0)} & M^{(0,1)} \\ M^{(1,0)} & M^{(1,1)} \end{bmatrix}, \quad N = \begin{bmatrix} N^{(0,0)} & N^{(0,1)} \\ N^{(1,0)} & N^{(1,1)} \end{bmatrix},$$

and

$$P = \begin{bmatrix} P^{(0,0)} & P^{(0,1)} \\ P^{(1,0)} & P^{(1,1)} \end{bmatrix}.$$

Each symbol such as $M^{(0,1)}$ now represents a complete 2×2 matrix block rather than one scalar element. The multiplication rule itself does not change. The same row-by-column rule can be applied at the level of tiles:

$$P^{(b_y, b_x)} = \sum_{ph} M^{(b_y, ph)} N^{(ph, b_x)}$$

Here b_y and b_x select one output tile, while ph moves along the shared dimension. For the 4×4 , $T = 2$ example,

$$\begin{aligned} P^{(0,0)} &= M^{(0,0)} N^{(0,0)} + M^{(0,1)} N^{(1,0)}, \\ P^{(0,1)} &= M^{(0,0)} N^{(0,1)} + M^{(0,1)} N^{(1,1)}, \\ P^{(1,0)} &= M^{(1,0)} N^{(0,0)} + M^{(1,1)} N^{(1,0)}, \\ P^{(1,1)} &= M^{(1,0)} N^{(0,1)} + M^{(1,1)} N^{(1,1)}. \end{aligned}$$

This block formulation explains why tiled matrix multiplication naturally proceeds in **phases**. One output tile is not usually produced from one pair of input tiles. Instead, the algorithm repeatedly selects one tile from M and one matching tile from N , multiplies them, and accumulates the result into the same output tile.

If W is exactly divisible by T , the number of phases is

$$N_{\text{phase}} = \frac{W}{T}.$$

For arbitrary dimensions, the general expression is

$$N_{\text{phase}} = \left\lceil \frac{W}{T} \right\rceil.$$

At the level of one output element, the same transformation can be written as

$$P_{\text{row}, \text{col}} = \sum_{ph} \sum_{k=0}^{T-1} M_{\text{row}, phT+k} N_{phT+k, \text{col}}$$

when W is divisible by T . The original long dot product has therefore been split into several shorter dot products. The outer index ph selects a tile pair, while the inner index k traverses the shared dimension inside the current pair of tiles.

8.3 3. A Complete 4×4 Example with $T = 2$

Now consider the upper-left output tile

$$P^{(0,0)} = \begin{bmatrix} P_{0,0} & P_{0,1} \\ P_{1,0} & P_{1,1} \end{bmatrix}.$$

Because $W = 4$ and $T = 2$, this tile requires two phases:

$$P^{(0,0)} = \underbrace{M^{(0,0)} N^{(0,0)}}_{\text{phase 0}} + \underbrace{M^{(0,1)} N^{(1,0)}}_{\text{phase 1}}$$

In phase 0, the required input tiles are

$$M^{(0,0)} = \begin{bmatrix} M_{0,0} & M_{0,1} \\ M_{1,0} & M_{1,1} \end{bmatrix}, \quad N^{(0,0)} = \begin{bmatrix} N_{0,0} & N_{0,1} \\ N_{1,0} & N_{1,1} \end{bmatrix}.$$

The four output elements receive the phase-0 contributions

$$\begin{aligned}
P_{0,0}^{(\text{partial})} &= M_{0,0}N_{0,0} + M_{0,1}N_{1,0}, \\
P_{0,1}^{(\text{partial})} &= M_{0,0}N_{0,1} + M_{0,1}N_{1,1}, \\
P_{1,0}^{(\text{partial})} &= M_{1,0}N_{0,0} + M_{1,1}N_{1,0}, \\
P_{1,1}^{(\text{partial})} &= M_{1,0}N_{0,1} + M_{1,1}N_{1,1}.
\end{aligned}$$

The reuse is now explicit. For example, $M_{0,0}$ contributes to both $P_{0,0}$ and $P_{0,1}$, while $N_{0,0}$ contributes to both $P_{0,0}$ and $P_{1,0}$. Instead of fetching these values independently for each output element, a tiled algorithm can fetch them once into a shared working set and let several output calculations reuse them.

Phase 1 uses the next pair of tiles,

$$M^{(0,1)} = \begin{bmatrix} M_{0,2} & M_{0,3} \\ M_{1,2} & M_{1,3} \end{bmatrix}, \quad N^{(1,0)} = \begin{bmatrix} N_{2,0} & N_{2,1} \\ N_{3,0} & N_{3,1} \end{bmatrix}.$$

The same four output elements then receive

$$\begin{aligned}
P_{0,0} &+= M_{0,2}N_{2,0} + M_{0,3}N_{3,0}, \\
P_{0,1} &+= M_{0,2}N_{2,1} + M_{0,3}N_{3,1}, \\
P_{1,0} &+= M_{1,2}N_{2,0} + M_{1,3}N_{3,0}, \\
P_{1,1} &+= M_{1,2}N_{2,1} + M_{1,3}N_{3,1}.
\end{aligned}$$

After phase 1, all four dot products are complete. The key point is that tiling has not changed the mathematics. It has only reorganized one long dot product into a sequence of shorter tile-sized dot products so that the data needed by several outputs can be handled together.

8.4 4. Mapping Tiles onto CUDA Blocks and Threads

The block-matrix structure maps naturally onto the CUDA execution hierarchy. **One thread block is assigned one $T \times T$ output tile, and one thread is assigned one output element inside that tile.** A $T \times T$ block therefore contains T^2 threads and produces T^2 output values.

For the $T = 2$ example, one block contains four threads:

	$tx = 0$	$tx = 1$
$ty = 0$	$P_{0,0}$	$P_{0,1}$
$ty = 1$	$P_{1,0}$	$P_{1,1}$

For a general block, the global output coordinates of thread (ty, tx) are

$$\boxed{row = blockIdx.y \cdot T + ty}$$

and

$$\boxed{col = blockIdx.x \cdot T + tx.}$$

Thus `blockIdx.y` and `blockIdx.x` choose the output tile, while `threadIdx.y` and `threadIdx.x` choose one element inside that tile.

The complete matrices M and N remain in **global memory** because they are large. During one phase, however, the block needs only one $T \times T$ tile of M and one $T \times T$ tile of N . Those two temporary working sets are placed in **shared memory**. Each thread also owns one private accumulator, typically named `Pvalue`, which is normally kept in a **register**.

The resulting memory mapping is therefore

complete input matrices	→	global memory,
current reusable input tiles	→	shared memory,
one thread's partial output	→	register.

This is an important architectural point: the tile itself is not a special CUDA object. A tile is an algorithmic partition of a matrix. CUDA realizes that partition by assigning one output tile to a block and using the block's shared memory as the on-chip storage for the current pair of input tiles.

8.5 5. One Phase on the GPU: Cooperative Loading, Reuse, and Synchronization

One phase has a fixed execution pattern. Assume a $T \times T$ thread block. There are T^2 threads and each input tile also contains T^2 elements. The threads can therefore load the tiles cooperatively: thread (ty, tx) loads one element from the current M tile and one element from the current N tile.

For phase ph , the shared-memory entries are

$$Mds_{ty,tx} = M_{row, phT+tx}$$

and

$$Nds_{ty,tx} = N_{phT+ty, col}.$$

The first expression means that the phase index moves the M tile horizontally from left to right. The output row is fixed, phT selects the beginning column of the current tile, and tx selects one column inside that tile. The second expression means that the phase index moves the N tile vertically from top to bottom. The output column is fixed, phT selects the beginning row of the tile, and ty selects one row inside it.

After the cooperative loads, the block must execute

```
1 __syncthreads();
```

because a thread will generally consume shared-memory values written by other threads. For example, thread $(0,0)$ may need $Mds[0][1]$, which was loaded by thread $(0,1)$, and $Nds[1][0]$, which was loaded by thread $(1,0)$. The first barrier therefore guarantees that **all shared-memory writes for the current tiles are complete before any thread starts reading those tiles for arithmetic**.

Once the tile is complete, thread (ty, tx) computes the phase contribution to its own output element:

$$Pvalue += \sum_{k=0}^{T-1} Mds_{ty,k} Nds_{k,tx}$$

Here ty remains fixed, so the thread reads one row of the shared M tile. The value tx remains fixed, so the thread reads one column of the shared N tile. The inner index k walks through the shared dimension exactly as in ordinary matrix multiplication. The difference is that these repeated arithmetic operations now consume values from fast on-chip shared memory instead of repeatedly fetching them from global memory.

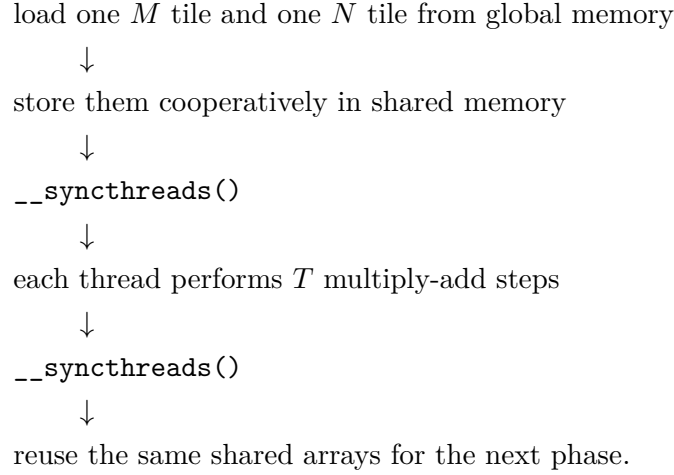
The partial result `Pvalue` is private to the thread and persists across all phases. After one phase, the thread does not write a partial result back to global memory. It simply keeps accumulating into the same register value until the complete dot product has been evaluated.

A second barrier is required after the arithmetic:

```
1 __syncthreads();
```

The shared arrays `Mds` and `Nds` are reused in the next phase. A fast thread must not overwrite them with the next tile pair while another thread is still reading the current tiles. The second barrier therefore guarantees that **all reads of the current tiles are complete before the shared-memory storage is overwritten**.

One phase can therefore be remembered as



This cooperative organization explains the reduction in global-memory traffic. A naive $T \times T$ block performs approximately

$$N_{\text{load,naive}} = 2WT^2$$

input element loads. In the tiled version there are W/T phases, and each phase loads only $2T^2$ elements, giving

$$\begin{aligned} N_{\text{load,tiled}} &= \frac{W}{T}(2T^2) \\ &= 2WT. \end{aligned}$$

The ideal reduction factor is therefore

$$\frac{N_{\text{load,naive}}}{N_{\text{load,tiled}}} = T$$

while the number of floating-point multiply-add operations is unchanged. Tiling therefore improves performance by reducing data movement and increasing computational intensity, not by reducing the mathematical amount of work.

8.6 6. From the Mapping to CUDA Code

Once the mathematical and architectural mapping is clear, the CUDA kernel is a direct translation. The following textbook-style version assumes square matrices and assumes that `Width` is divisible by `TILE_WIDTH`.

```

1 #define TILE_WIDTH 16
2
3 __global__ void matrixMultTiledKernel(
4     const float* M,
5     const float* N,
6     float* P,

```

```

7   int Width)
8   {
9       __shared__ float Mds[TILE_WIDTH][TILE_WIDTH];
10      __shared__ float Nds[TILE_WIDTH][TILE_WIDTH];
11
12      int tx = threadIdx.x;
13      int ty = threadIdx.y;
14
15      int row = blockIdx.y * TILE_WIDTH + ty;
16      int col = blockIdx.x * TILE_WIDTH + tx;
17
18      float Pvalue = 0.0f;
19
20      for (int ph = 0;
21           ph < Width / TILE_WIDTH;
22           ++ph)
23      {
24          Mds[ty][tx] =
25              M[row * Width
26                + ph * TILE_WIDTH
27                + tx];
28
29          Nds[ty][tx] =
30              N[(ph * TILE_WIDTH + ty)
31                * Width
32                + col];
33
34          __syncthreads();
35
36          for (int k = 0;
37               k < TILE_WIDTH;
38               ++k)
39          {
40              Pvalue +=
41                  Mds[ty][k]
42                  * Nds[k][tx];
43          }
44
45          __syncthreads();
46      }
47
48      P[row * Width + col] = Pvalue;
49  }

```

The first part of the kernel establishes the output mapping. The variables `tx` and `ty` are the thread coordinates inside the block, while

$$row = blockIdx.y \cdot T + ty, \quad col = blockIdx.x \cdot T + tx$$

identify the output element computed by that thread. The accumulator `Pvalue` is initialized once and remains private to the thread across all phases.

The outer loop

```

1  for (int ph = 0;
2      ph < Width / TILE_WIDTH;
3      ++ph)

```

implements the block-matrix sum

$$P^{(b_y, b_x)} = \sum_{ph} M^{(b_y, ph)} N^{(ph, b_x)}.$$

Within one phase, the M loading expression

```

1 Mds[ty][tx] =
2   M[row * Width
3     + ph * TILE_WIDTH
4     + tx];

```

is the row-major one-dimensional form of

$$Mds_{ty,tx} = M_{row, phT+tx}.$$

Likewise,

```

1 Nds[ty][tx] =
2   N[(ph * TILE_WIDTH + ty)
3     * Width
4     + col];

```

is the row-major form of

$$Nds_{ty,tx} = N_{phT+ty, col}.$$

These two statements are not arbitrary indexing tricks. They are the direct element-wise realization of loading the tile pair $M^{(b_y, ph)}$ and $N^{(ph, b_x)}$ required by the current block-matrix product.

After the first synchronization barrier, the inner loop

```

1 for (int k = 0;
2     k < TILE_WIDTH;
3     ++k)
4 {
5     Pvalue +=
6         Mds[ty][k]
7         * Nds[k][tx];
8 }

```

computes one tile-sized partial dot product. The relationship between the original dot-product index q and the tiled indices is

$$q = phT + k.$$

Thus the original loop over $q = 0, \dots, W - 1$ has been split into an outer phase loop and an inner tile loop. The mathematics is unchanged; only the organization of data movement and reuse has changed.

After every phase has finished, the thread owns the complete result

$$Pvalue = P_{row, col},$$

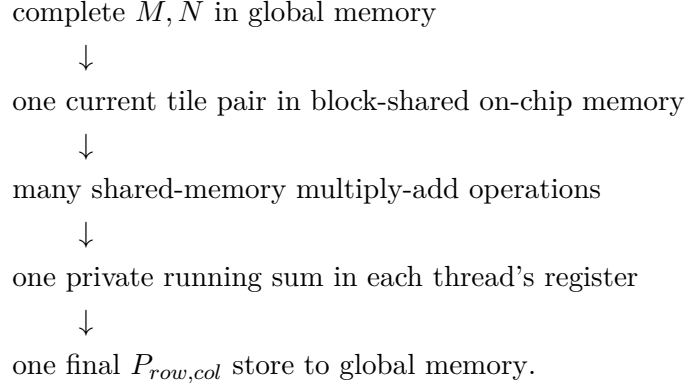
and performs one final global-memory store:

```

1 P[row * Width + col] = Pvalue;

```

The complete architectural data path is therefore



The central lesson is therefore not merely that a tiled kernel uses `__shared__` arrays. **Tiling is an algorithmic reorganization that makes the CUDA memory hierarchy match the reuse structure already present in matrix multiplication.** Large matrices stay in global memory, a block stages the current reusable working set in shared memory, and each thread keeps its own accumulator in registers. This reduces repeated global-memory traffic and allows each fetched value to support more useful arithmetic.

9 Boundary Checks for Arbitrary Matrix Sizes

The basic tiled kernel assumes that the matrix dimensions are exact multiples of `TILE_WIDTH`. For arbitrary sizes, threads near the matrix boundaries may attempt to load nonexistent input elements or write nonexistent output elements. Every global-memory access therefore needs its own boundary condition.

For a general matrix multiplication

$$A_{M \times K} B_{K \times N} = C_{M \times N},$$

define

$$row = blockIdx.y \cdot T + ty, \quad col = blockIdx.x \cdot T + tx,$$

and, in phase ph ,

$$A_{col} = phT + tx, \quad B_{row} = phT + ty.$$

The cooperative loads should then be guarded as follows:

```

1 if (row < M && A_col < K)
2     Ads[ty][tx] = A[row * K + A_col];
3 else
4     Ads[ty][tx] = 0.0f;
5
6 if (B_row < K && col < N)
7     Bds[ty][tx] = B[B_row * N + col];
8 else
9     Bds[ty][tx] = 0.0f;
```

Invalid input positions are filled with zero so that the same shared-memory dot-product loop can still be used for an incomplete tile. The final output store is also guarded:

```

1 if (row < M && col < N)
2     C[row * N + col] = Pvalue;
```

A thread that does not produce a valid output element should not simply return before the block synchronization points, because it may still be responsible for loading shared-memory data needed by other threads. Boundary checks must therefore be applied to the individual memory accesses rather than by removing such threads from the tiled execution.

For arbitrary K , the number of phases is

$$N_{\text{phase}} = \left\lceil \frac{K}{T} \right\rceil$$

which can be implemented using integer arithmetic as

```
1 int num_phases =
2   (K + TILE_WIDTH - 1) / TILE_WIDTH;
```

10 Shared-Memory Usage and Occupancy

Shared memory is fast, but it is also a limited per-SM hardware resource. If every block requires a large amount of shared memory, fewer blocks can reside simultaneously on one SM, which may reduce occupancy and the GPU's ability to hide latency.

For the basic tiled matrix-multiplication kernel with tile width T , two shared-memory tiles are allocated:

$$Mds[T][T], \quad Nds[T][T].$$

For single-precision data, the shared-memory requirement per block is

$$S_{\text{block}} = 2T^2 \times 4 \text{ bytes.}$$

For $T = 16$,

$$S_{\text{block}} = 2 \times 16^2 \times 4 = 2048 \text{ bytes.}$$

This is relatively small, so the simple 16×16 tiled kernel is usually not limited by shared memory alone. In general, however, occupancy can be constrained jointly by shared memory, registers, the maximum number of resident warps, the maximum number of threads, and the maximum number of blocks per SM. Higher occupancy is therefore not automatically equivalent to higher performance; the important goal is to maintain enough active warps to hide latency while preserving efficient data reuse.

The declarations used so far allocate shared memory statically:

```
1 __shared__ float Mds[TILE_WIDTH][TILE_WIDTH];
2 __shared__ float Nds[TILE_WIDTH][TILE_WIDTH];
```

Their sizes are fixed at compile time. CUDA also supports **dynamic shared memory**, whose size is selected when the kernel is launched:

```
1 extern __shared__ float shared[];
2
3 float* Mds = shared;
4 float* Nds = shared
5     + TILE_WIDTH * TILE_WIDTH;
```

The host supplies the required number of bytes as the third kernel-launch configuration argument:

```
1 size_t shared_bytes =  
2     2 * TILE_WIDTH * TILE_WIDTH  
3     * sizeof(float);  
4  
5 kernel<<<grid, block, shared_bytes>>>(...);
```

Dynamic shared memory is useful when the amount of per-block scratch space should be chosen at runtime according to the kernel configuration or the resources available on a particular GPU. The broader principle is that **shared memory improves locality and reuse, but its capacity must be balanced against the number of blocks and warps that can remain resident on each SM.**